PYTHON MASTERY ROADMAP

Beginner to Advanced

Python fundamentals • Software engineering • Backend • APIs • Full-stack development • Data science • Machine learning • Deployment

A project-based, 30-week learning plan with practical milestones, resources, and a capstone path.

How to Use This Roadmap

The goal is not just to learn Python syntax. It is to become capable of building complete applications, designing reliable backends, integrating APIs, training machine-learning models, and deploying real-world projects.

Recommended pace: about 30 weeks at 10–15 hours per week. Beginners may need longer, especially if programming logic, mathematics, or web development is new. The weeks are a suggested sequence, not deadlines.

A useful long-term capstone is MOBIUS AI: a mobility-focused application that predicts traffic congestion. It can connect data analysis, machine learning, APIs, databases, frontend development, and deployment.

Roadmap at a Glance

1. Phase 1 — Python fundamentals (Weeks 1–4): variables, data types, conditions, loops, collections, functions, and problem-solving.
1. Phase 2 — Intermediate Python (Weeks 5–7): files, exceptions, modules, OOP, iterators, generators, decorators, and standard libraries.
1. Phase 3 — Advanced Python and software engineering (Weeks 8–9): testing, typing, clean code, algorithms, memory, concurrency, and async concepts.
1. Phase 4 — Backend development (Weeks 10–13): web fundamentals, FastAPI, SQL, PostgreSQL, authentication, security, and tests.
1. Phase 5 — API engineering (Weeks 14–15): consuming and designing APIs, retries, webhooks, async work, queues, and WebSockets.
1. Phase 6 — Full-stack development (Weeks 16–19): HTML, CSS, JavaScript, React, integration, and deployment.
1. Phase 7 — Data science and machine learning (Weeks 20–25): mathematics, NumPy, pandas, ML, evaluation, deep learning, time series, and geospatial data.
1. Phase 8 — Serve ML through APIs (Weeks 26–27): model artifacts, inference endpoints, monitoring, and integration.
1. Phase 9 — Production engineering (Weeks 28–29): Linux, Docker, CI/CD, cloud deployment, monitoring, and system design.
1. Phase 10 — Specialization and portfolio (Week 30 onward): deepen backend, full-stack, ML engineering, or applied AI skills.

1. Set Up Your Learning Environment

1. Python 3, using a current stable release.
1. VS Code for writing code.
1. A terminal for running programs and commands.
1. Git and GitHub for version control and storing projects.
1. A virtual environment for each Python project.
1. Jupyter Notebook later, when you reach data science and machine learning.

Learn to run a Python file, install a package with pip, create a virtual environment, and read error messages.

```
python --version
python -m venv .venv
```

Activate the environment on Windows:

```
.venv\Scripts\Activate.ps1
```

Create main.py:

```
name = input("What is your name? ")
print(f"Hello, {name}!")
```

Run it:

```
python main.py
```

Milestone: write, run, debug, and modify a program without needing a tutorial for every step.

2. Phase 1 — Python Fundamentals (Weeks 1–4)

Do not rush this phase. Every advanced topic depends on understanding how programs store information, make decisions, repeat actions, and organize logic.

Week 1: Syntax, Variables, and Data Types

- What Python is, how interpreted languages work, and how a program executes.
- Syntax, indentation, comments, statements, and expressions.
- Variables, assignment, naming conventions, and reserved keywords.
- Primitive data types: int, float, str, and bool.
- Type conversion: int(), float(), str(), bool().
- Operators: arithmetic, comparison, logical, assignment, membership, and identity.
- Input and output using input() and print().
- String formatting with f-strings.
- Basic debugging and understanding tracebacks.

Example:

```
product = "Laptop"
price = 2500.00
quantity = 2

total = price * quantity
print(f"{product}: AED {total:.2f}")
```

Practice: temperature converter; circle-area calculator; total price after discount; convert seconds into hours/minutes/seconds; calculate age from birth year.

Milestone: write small programs from scratch using variables, input, arithmetic, and formatted output.

Week 2: Conditions and Loops

- if, elif, and else; nested conditions and Boolean expressions.
- Truthy and falsy values.
- for loops and while loops.
- range(), break, continue, and pass.
- Nested loops and basic algorithmic thinking.
- Avoiding infinite loops and off-by-one errors.

```
score = int(input("Enter your score: "))

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "Needs improvement"

print(f"Grade: {grade}")
```

Practice: number guessing game; multiplication table generator; prime number checker; FizzBuzz; password-attempt simulator; simple ATM menu.

Milestone: solve problems involving several conditions, repeated actions, and user input.

Week 3: Data Structures and Collections

Structure	What to master
Lists	Indexing, slicing, appending, inserting, removing, sorting
Tuples	Immutability, unpacking, returning multiple values
Dictionaries	Keys, values, nested dictionaries, updating and lookup
Sets	Uniqueness, membership, unions, intersections
Strings	Slicing, searching, splitting, joining, replacement
Nested collections	Lists of dictionaries, dictionaries of lists

- Mutable vs. immutable objects; references and object identity.

- Indexing and slicing; membership testing using in.
- List, dictionary, and set comprehensions.
- Sorting with key= and choosing the appropriate collection.

```
students = [
    {"name": "Ali", "score": 85},
    {"name": "Sara", "score": 94},
    {"name": "Omar", "score": 78},
]

top_student = max(students, key=lambda s: s["score"])
print(top_student["name"])
```

Practice projects: contact book; student grade manager; inventory tracker; word-frequency counter; shopping cart calculator.

Milestone: choose the right data structure without defaulting to lists for everything.

Week 4: Functions and Problem-Solving

- Defining and calling functions; parameters and arguments.
- Positional and keyword arguments; default parameters; return values.
- Local and global scope.
- *args and **kwargs.
- Docstrings; pure functions and side effects.
- Decomposing a large problem into smaller functions.
- Recursion and base cases; basic algorithm analysis.

```
def calculate_total(price, quantity, discount=0):
    subtotal = price * quantity
    return subtotal * (1 - discount)

total = calculate_total(100, 3, 0.10)
print(total)
```

Learn the difference between printing a result and returning a result. This becomes essential for APIs, tests, and composing larger applications.

Phase 1 final project: command-line expense tracker with adding/listing expenses, categories and amounts, totals by category, input validation, a menu-driven interface, and functions for each major operation.

3. Phase 2 — Intermediate Python (Weeks 5–7)

Week 5: Files, Exceptions, and Modules

- File handling: reading and writing text files; modes r, w, a, x.
- The with statement and context managers.
- CSV and JSON; paths and directories with pathlib; UTF-8 encoding.
- Basic file-processing automation.
- Exception handling: try, except, else, finally.
- Common errors: ValueError, TypeError, KeyError, FileNotFoundError.
- Raising exceptions with raise; custom exception classes.
- Expected input errors vs. programming bugs.
- Imports and namespaces; creating modules and packages; __init__.py.
- The if __name__ == '__main__': pattern.
- Installing third-party libraries with pip; virtual environments and dependency management.

Project: upgrade the expense tracker to save data in JSON or CSV and restore it when the application restarts.

Week 6: Object-Oriented Programming (OOP)

- Classes and objects; attributes and instance methods.
- Constructors with __init__; instance variables and class variables.

- Encapsulation and properties.
- Inheritance, method overriding, and polymorphism.
- Composition vs. inheritance.
- Abstract base classes and interfaces.
- Special methods: __str__, __repr__, and __eq__.
- Dataclasses and enums.
- When OOP helps—and when functions and dictionaries are simpler.

```
from dataclasses import dataclass

@dataclass
class Expense:
    description: str
    amount: float
    category: str

    def is_large(self):
        return self.amount >= 500
```

Project: inventory management system with Product, Inventory, and Order models.

Week 7: Pythonic Programming and Standard Libraries

- List, set, and dictionary comprehensions.
- Iterables, iterators, the iterator protocol, generators, and yield.
- Generator expressions; lambda functions and custom sorting keys.
- enumerate(), zip(), any(), all(), map(), and filter().
- Unpacking and the * operator.
- Decorators and closures.
- Type hints.
- Standard libraries: collections, itertools, functools, datetime, decimal, re, statistics, and logging.

Understand the difference between an iterable and an iterator, why generators save memory, and how decorators wrap existing functions.

Project: log-file analyzer that reads a large file, extracts errors, groups events by date, and produces a report.

4. Phase 3 — Advanced Python and Software Engineering (Weeks 8–9)

Week 8: Testing, Typing, and Clean Architecture

- Unit tests with pytest; assertions and test fixtures.
- Parametrized tests; mocking external services; integration tests.
- Edge cases and boundary conditions.
- Type annotations for functions, collections, and return values.
- TypedDict, Protocol, and generics at a practical level.
- Static type checking with mypy or Pyright.
- Formatting and linting with Ruff.
- Debugging with breakpoints and a debugger.
- Environment variables and configuration files.
- Secrets management: never commit API keys or passwords.
- Refactoring, separation of concerns, and dependency injection.
- Git fundamentals: branching, commits, pull requests, and resolving conflicts.

Professional habit: important business logic should be testable without starting the entire application.

Week 9: How Python Works Under the Hood

- [ ] Names, objects, references, and object identity.
- [ ] Shallow copy vs. deep copy.
- [ ] Mutable default argument pitfalls.
- [ ] LEGB name resolution and closures.
- [ ] Python's data model and special methods; hashability and equality.
- [ ] Time and space complexity; Big O notation.
- [ ] Data structures and algorithm patterns.
- [ ] Recursion, stacks, queues, linked lists, trees, and graphs.
- [ ] Sorting and searching algorithms; hash tables and sets.
- [ ] Memory management and garbage collection.
- [ ] The Global Interpreter Lock (GIL), conceptually.
- [ ] Concurrency vs. parallelism; threading, multiprocessing, and asyncio.
- [ ] Context managers and decorators in more depth.

```
# Avoid this
def add_item(item, items=[]):
    items.append(item)
    return items

# Better
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items
```

The first version reuses the same list across calls because default arguments are evaluated when the function is defined.

Projects: reusable Python package with documentation and tests; concurrent URL checker; log-processing utility; command-line application with clean module boundaries.

You do not need every CPython implementation detail before backend development. Learn enough to write good code and investigate performance issues intelligently.

5. Phase 4 — Backend Development with Python (Weeks 10–13)

Week 10: Web Fundamentals

- [ ] Client-server architecture; browsers, servers, DNS, and domains.
- [ ] HTTP and HTTPS; request and response structure.
- [ ] Methods: GET, POST, PUT, PATCH, DELETE.
- [ ] Status codes: 200, 201, 204, 400, 401, 403, 404, 409, 422, 429, and 500.
- [ ] Headers, cookies, query parameters, and request bodies.
- [ ] JSON and serialization; REST architecture and resource-oriented URLs.
- [ ] Stateless requests and sessions.
- [ ] CORS and why browsers enforce it.
- [ ] Basic networking concepts, ports, and localhost.

```
GET /api/products/42 HTTP/1.1
Host: example.com
Accept: application/json
```

The client requests product 42. The backend processes the request and returns a status code, headers, and possibly a JSON response.

Week 11: FastAPI

Recommendation: start with FastAPI. It has a straightforward learning curve, integrates well with type hints, and suits APIs that later serve machine-learning models.

1. Creating a FastAPI application and running a development server.
1. Path operations and route parameters.
1. Query parameters and request bodies.
1. Pydantic models and data validation.
1. Response models and serialization.
1. Status codes and error handling.
1. Dependency injection and middleware.
1. Automatic OpenAPI documentation.
1. File uploads and form data.
1. Async endpoints and await.
1. Configuration with environment variables.
1. Application lifespan and startup/shutdown tasks.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class TaskCreate(BaseModel):
    title: str
    completed: bool = False

tasks = []

@app.get("/tasks")
def list_tasks():
    return tasks

@app.post("/tasks", status_code=201)
def create_task(task: TaskCreate):
    tasks.append(task.model_dump())
    return task
```

This is a learning example, not a production database-backed application. The data disappears when the server restarts.

Project: task-management API with CRUD (create, read, update, delete), validation, and automatic API documentation.

Week 12: SQL and Databases

Learn SQL before relying heavily on an ORM.

1. Tables, rows, columns, and data types.
1. Primary and foreign keys.
1. SELECT, INSERT, UPDATE, DELETE; WHERE; sorting and LIMIT.
1. Aggregate functions and GROUP BY.
1. Joins: inner, left, and other common join patterns.
1. Constraints and unique values; indexes and query performance.
1. Transactions and ACID properties; database normalization.
1. Migrations and schema changes.
1. PostgreSQL database creation, connection, table inspection, indexes, and transactions.
1. SQLAlchemy 2.x: ORM models, relationships, sessions, and transaction boundaries.
1. Alembic migrations; connection pooling.
1. Parameterized queries and SQL injection prevention; database testing.

Use PostgreSQL as your main relational database for serious projects. SQLite is useful for small experiments and some lightweight applications.

Project: replace the task API's in-memory list with PostgreSQL. Add users, task ownership, database relationships, and pagination.

Week 13: Authentication, Security, and API Quality

- [ ] Authentication vs. authorization.
- [ ] Password hashing with an appropriate library.
- [ ] Session-based authentication and token-based authentication.
- [ ] OAuth 2.0 and OpenID Connect concepts.
- [ ] JWT structure, validation, expiration, and revocation trade-offs.
- [ ] Role-based and resource-based permissions.
- [ ] CORS configuration and rate limiting.
- [ ] Input validation and output filtering.
- [ ] Secure secret storage.
- [ ] SQL injection, XSS, CSRF, and common web risks.
- [ ] API versioning; pagination, filtering, sorting, and consistent errors.
- [ ] Logging and request IDs; unit and integration testing.
- [ ] Idempotency and retry behavior.

Project: secure task-management backend with registration, login, protected routes, ownership checks, database persistence, tests, and API documentation.

Do not implement password storage or authentication cryptography from scratch. Use well-maintained libraries and current security guidance.

6. Phase 5 — API Engineering and External Integrations (Weeks 14–15)

Week 14: Consuming and Designing APIs

- [ ] REST API design conventions; JSON requests and responses.
- [ ] Calling APIs with requests or httpx.
- [ ] Authentication headers and API keys.
- [ ] Timeouts, retries, and exponential backoff.
- [ ] Handling HTTP errors and rate limits.
- [ ] Pagination and cursor-based pagination.
- [ ] Webhooks and event-driven communication.
- [ ] Idempotency keys; API documentation with OpenAPI.
- [ ] API testing with pytest and HTTPX; mocking external API responses.
- [ ] Environment-based configuration and basic caching.

Project: API integration service that retrieves information from an external API, validates responses, caches appropriate results, and exposes a clean endpoint of its own.

Week 15: Asynchronous APIs and Background Jobs

- [ ] Synchronous vs. asynchronous execution; async def, await, and asyncio.
- [ ] Async HTTP requests with HTTPX.
- [ ] When to use threads, processes, or asynchronous I/O.
- [ ] Background tasks and their limitations.
- [ ] Task queues with Celery or RQ.
- [ ] Redis for caching and coordination.
- [ ] Message queues and event-driven architecture.
- [ ] WebSockets and real-time communication; server-sent events.

1 Scheduled jobs and periodic tasks.

Important distinction: asyncio helps with concurrent I/O; it does not automatically make CPU-heavy machine-learning computations faster.

Project: data collection service that calls several APIs concurrently, handles failures, stores results, and exposes collected data through FastAPI.

7. Phase 6 — Full-Stack Development (Weeks 16–19)

Python is commonly used for the backend, while JavaScript or TypeScript powers the browser frontend. You do not need a different backend language to build a modern full-stack application.

Application Architecture

1 Frontend — browser: HTML, CSS, JavaScript, React.

1 HTTP requests and JSON responses connect the frontend to the backend.

1 Backend — Python: FastAPI, validation, business logic, authentication.

1 Data and external services: PostgreSQL, Redis, external APIs, ML models.

Week 16: HTML and CSS

HTML: document structure and semantic elements; headings, paragraphs, links, lists, and images; forms, labels, inputs, and buttons; tables and accessible markup.

CSS: selectors, specificity, inheritance, and cascade; box model, margins, padding, and borders; display, positioning, and overflow; Flexbox and CSS Grid; responsive design and media queries; typography, colors, reusable styling, CSS variables, and browser developer tools.

Project: create a responsive landing page and dashboard layout without a component framework.

Week 17: JavaScript Fundamentals

1 Variables with const and let; strings, numbers, arrays, and objects.

1 Conditions, loops, and functions; scope and closures.

1 Array methods: map, filter, and reduce.

1 Destructuring and spread syntax; modules and imports.

1 DOM manipulation and event listeners.

1 Promises and error handling; async/await.

1 Fetching data with fetch(); JSON parsing and browser developer tools.

1 TypeScript basics after becoming comfortable with JavaScript.

Project: interactive frontend that fetches and displays tasks from your FastAPI backend.

Week 18: React

1 Components and JSX; props and component composition.

1 State with useState; side effects with useEffect.

1 Handling forms and validation; conditional rendering and lists.

1 Controlled inputs and custom hooks.

1 Loading, success, empty, and error states.

1 API integration and data fetching; routing.

1 Shared state and context; component testing.

1 Basic accessibility; TypeScript with React.

Choose one styling approach—plain CSS, CSS Modules, or Tailwind CSS—instead of learning several at once.

Project: React dashboard with search, filters, forms, and API-backed data.

Week 19: Full-Stack Integration and Deployment

1 Connecting React to FastAPI; handling CORS correctly.

1. Authentication across frontend and backend.
1. Managing frontend and backend environment variables.
1. Form validation on both sides.
1. Pagination, sorting, and filtering; uploading files.
1. Error handling and loading states.
1. Production builds; hosting frontend and backend separately.
1. Domain names and HTTPS; database deployment and migrations.
1. Logs, monitoring, and debugging production issues.

Final project: complete application with React frontend, FastAPI backend, PostgreSQL database, user authentication, and deployment.

8. Phase 7 — Mathematics, Data Analysis, and Machine Learning (Weeks 20–25)

Week 20: Mathematics for Machine Learning

Subject	Topics to learn
Algebra	Equations, functions, exponents, logarithms
Statistics	Mean, median, variance, standard deviation, percentiles
Probability	Conditional probability, distributions, expected value
Linear algebra	Vectors, matrices, dot products, matrix multiplication
Calculus	Derivatives, gradients, chain rule
Optimization	Loss functions, gradient descent, learning rates

Learn concepts alongside small coding exercises. Prioritize probability, statistics, and vectors first; deeper calculus matters more in neural networks and deep learning.

Week 21: NumPy, pandas, and Data Visualization

NumPy: arrays, dimensions, shapes, and data types; indexing, slicing, and Boolean masks; vectorization and broadcasting; aggregation and statistical functions; matrix operations; random number generation; missing values and numerical precision.

pandas: Series and DataFrames; loading CSV, Excel, JSON, and database data; selecting rows and columns; filtering, sorting, and transforming data; missing values and duplicates; groupby, aggregation, and pivot tables; merging and joining; dates, timestamps, and time-series data; data cleaning and validation.

Visualization: Matplotlib; Seaborn as a later option; line charts, bar charts, histograms, scatter plots, and box plots; detecting outliers, trends, and unusual patterns.

Project: analyze a traffic dataset to identify peak hours, congestion hotspots, average speeds, and day-of-week patterns.

Week 22: Machine-Learning Foundations

1. Supervised vs. unsupervised learning; regression vs. classification.
1. Features, targets, and labels; training, validation, and test datasets.
1. Baseline models.
1. Linear regression and logistic regression.
1. Decision trees and random forests.
1. Gradient boosting at a conceptual and practical level.
1. K-nearest neighbors; clustering with K-means.
1. Feature scaling and encoding categorical variables.
1. Pipelines and column transformers.
1. Cross-validation and hyperparameter tuning.

1. Data leakage and overfitting.
1. Model persistence and reproducibility.

Project: predict traffic speed or travel time from historical observations and features such as hour, day of week, location, and weather.

Week 23: Model Evaluation and Feature Engineering

Regression metrics: MAE (mean absolute error), MSE, RMSE, and R^2, interpreted with care.

Classification metrics: accuracy, precision, recall, F1-score, confusion matrices, and ROC-AUC when appropriate.

Feature engineering: missing values, scaling numerical features, encoding categories, extracting hour/weekday/holiday features, rolling averages and lag features, feature selection, and avoiding target leakage.

Time-series evaluation: chronological train/test splits, rolling or expanding-window validation, seasonal patterns and trends, baselines such as latest observation or historical average, forecast horizons, and prediction intervals.

For traffic forecasting, do not randomly mix future observations into training. Test procedures should reflect predicting future conditions from past data.

Project: compare a simple baseline against two or three models, evaluate errors on future dates, and document where predictions fail.

Week 24: Advanced Machine Learning and Deep Learning

Classical ML extensions: XGBoost, LightGBM, or similar gradient-boosting methods; imbalanced datasets; feature importance and permutation importance; model calibration; explainability with SHAP when useful; unsupervised anomaly detection.

Deep learning foundations: neural networks and layers; activation functions; loss functions; forward propagation and backpropagation; gradient descent and optimizers; regularization and dropout; training loops and validation; GPU concepts.

Choose a framework such as PyTorch. Study convolutional networks, recurrent networks, attention, and transformers when you have a specific reason to use them.

Project: compare a tree-based traffic prediction model with a simple neural network. Keep the neural network only if it meaningfully improves results.

Week 25: Time Series, Geospatial Data, and Applied AI

1. Time-series forecasting and temporal validation.
1. Autocorrelation and seasonal patterns; lagged variables and rolling statistics.
1. Forecasting multiple time steps ahead.
1. Geospatial coordinates and map projections.
1. Distance calculations and spatial joins.
1. GeoPandas and, when appropriate, PostGIS.
1. Mapping roads to segments and locations.
1. Graphs and networks: nodes, edges, routes, and connectivity.
1. Spatial and temporal aggregation.
1. External variables such as weather, incidents, and events.
1. Data drift and prediction monitoring.

For MOBIUS AI, a useful first target is predicting road-segment congestion 15–30 minutes ahead; choose the horizon based on data availability and the demo's needs.

You do not need deep learning or advanced graph neural networks for a credible first prototype. Start with historical data, a baseline forecast, and a well-evaluated model.

9. Phase 8 — Serve Machine-Learning Models Through APIs (Weeks 26–27)

1. Saving and loading trained models.
1. Keeping preprocessing consistent between training and inference.
1. Model versioning and reproducible artifacts.

- Prediction endpoints with FastAPI; validating features with Pydantic.
- Returning predictions, confidence information, and timestamps.
- Batch predictions vs. real-time inference.
- Separating model training from API serving.
- Background jobs for expensive computations.
- Caching repeated requests when appropriate.
- Input drift and prediction monitoring.
- Logging latency, failures, and model versions.
- Protecting the API from invalid or excessive requests.
- Testing predictions and API contracts.

```
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()

class TrafficInput(BaseModel):
    hour: int = Field(ge=0, le=23)
    current_speed: float = Field(gt=0)
    vehicle_count: int = Field(ge=0)

@app.post("/predict")
def predict_traffic(data: TrafficInput):
    # Replace with a trained, loaded model.
    # This placeholder is not a real prediction.
    return {
        "status": "model_not_connected",
        "input": data.model_dump()
    }
```

Separate API mechanics from model logic. Train and evaluate the model independently, then load it in a service that validates inputs and returns predictions.

10. Phase 9 — Production Engineering and Deployment (Weeks 28–29)

DevOps and Deployment

- Linux and shell fundamentals.
- Environment variables and configuration management.
- Docker images and containers; Dockerfiles and Docker Compose.
- Containerizing FastAPI applications.
- PostgreSQL and Redis service configuration.
- CI/CD using GitHub Actions; automated testing before deployment.
- Cloud application hosting.
- Database migrations in deployment workflows.
- HTTPS, domains, and reverse proxies.
- Secrets and permissions.
- Logs, health checks, metrics, and alerts.
- Backups and recovery basics.

Backend Architecture and System Design

- Modular monoliths and service boundaries.
- Separating business logic from transport and storage.
- Repository and service layers when justified.
- Database indexing and query optimization.

1. Caching strategies; queues and asynchronous workloads.
1. Rate limiting and resource limits.
1. Horizontal vs. vertical scaling.
1. Reliability, retries, timeouts, and graceful failure.
1. Load testing and performance profiling.
1. Basic threat modeling and operational documentation.

Project: deploy your full-stack ML application with Docker, automated tests, a managed database, environment-based secrets, and basic monitoring.

Avoid jumping straight into microservices and Kubernetes. A well-designed monolith is generally a better first production project for a solo developer.

11. Phase 10 — Advanced Specialization and Portfolio (Week 30 Onward)

Backend Engineering

Master advanced SQL, transactions, distributed-systems concepts, caching, async workloads, API design, profiling, security, and system design. Portfolio: a secure, tested, scalable API service.

Full-Stack Engineering

Deepen React and TypeScript, application state, accessibility, frontend testing, performance, security, and production architecture. Portfolio: a complete user-facing SaaS-style application.

Machine-Learning Engineering

Study advanced forecasting, model deployment, experiment tracking, data pipelines, model monitoring, and reproducible training. Portfolio: a model-backed prediction service with documented evaluation.

AI Agents and Applied AI

Learn tool calling, structured outputs, retrieval-augmented generation (RAG), embeddings, evaluation, workflow orchestration, and agent safety. Portfolio: an agent that calls your API, analyzes data, and recommends actions.

You do not need to pursue all four specializations equally. Learn shared foundations, then deepen the area most relevant to your goals.

12. Weekly Study Schedule

If you can study for 2 hours a day, 6 days a week, you will have approximately 12 focused hours per week.

Activity	Time
Learn a concept from a lesson or book	25 minutes
Write code and follow examples	35 minutes
Solve exercises without looking at solutions	35 minutes
Review mistakes, take notes, and commit to Git	15 minutes
Total focused routine	110 minutes

Use the remaining 10 minutes for a break or to plan the next session.

1. Day 1: learn new concepts.
1. Day 2: practice small examples.
1. Day 3: solve exercises without tutorials.
1. Day 4: study the next set of concepts.
1. Day 5: apply concepts to a project.
1. Day 6: debug, test, revise, and document.

1. Day 7: rest or catch up.

Do not move on just because you finished watching a course. Move on when you can apply the concepts yourself.

13. Recommended Learning Resources

Python fundamentals: Python Tutorial — Useful reference; assumes some programming familiarity. https://docs.python.org/3/tutorial/

Beginner-friendly Python: Python for Everybody — Gentler introduction if you have never programmed. https://www.py4e.com/

Structured Python practice: Harvard CS50's Introduction to Programming with Python — Use problem sets to test independent problem-solving. https://cs50.harvard.edu/python/

Git and version control: Pro Git book — Commits, branches, merging, and recovering from mistakes. https://git-scm.com/book/en/v2

Backend and APIs: FastAPI documentation — Follow examples while building your own API. https://fastapi.tiangolo.com/

Database ORM: SQLAlchemy 2.x documentation — ORM models, relationships, and database sessions. https://docs.sqlalchemy.org/en/20/

SQL database: PostgreSQL documentation — SQL, indexes, transactions, and administration. https://www.postgresql.org/docs/

Web foundations: MDN Web Development — HTML, CSS, and JavaScript. https://developer.mozilla.org/en-US/docs/Learn_web_development

Frontend: React Learn — Learn React after JavaScript fundamentals. https://react.dev/learn

NumPy: NumPy learning resources — Array programming and numerical computing. https://numpy.org/learn/

pandas: pandas getting started — DataFrames and data manipulation. https://pandas.pydata.org/docs/getting_started/

Machine learning: scikit-learn getting started — Classical ML workflows. https://scikit-learn.org/stable/getting_started.html

Hands-on practice: Kaggle Learn — Short lessons and practical data exercises. https://www.kaggle.com/learn

Deployment: Docker getting started — Containers and images. https://docs.docker.com/get-started/

Automation: GitHub Actions documentation — Automated testing and deployment workflows. https://docs.github.com/en/actions

14. Capstone Project — MOBIUS AI

Build incrementally rather than attempting the entire platform at the beginning.

1. Python foundation: read traffic observations and calculate average speed, traffic volume, and congestion levels.
2. Data analysis: use pandas to clean historical traffic data, analyze peak hours, and visualize congestion hotspots.
3. Machine learning: train a baseline model that predicts traffic speed or congestion for a future time window.
4. Backend: create FastAPI endpoints such as /roads, /traffic/current, and /predictions.
5. Database: store road segments, timestamps, observations, and predictions in PostgreSQL.
6. Frontend: build a React dashboard with a map, congestion indicators, and a forecast timeline.
7. AI-agent layer: let an agent inspect predicted congestion, query relevant data, and propose interventions. Keep recommendations advisory at first rather than automatically changing real traffic infrastructure.
8. Deployment: package and deploy the application; demonstrate predictions with a repeatable scenario and measured results.

You can start with an openly available or simulated dataset. Privileged access to live city traffic infrastructure is not required to demonstrate the core prediction pipeline.

What to Measure

1. Forecast MAE against a simple baseline.
1. Accuracy at predicting high-congestion periods.
1. API response latency.
1. Forecast coverage across road segments.
1. Reliability when input data is missing or an external service fails.
1. Number of false congestion alerts.

1. Effect of proposed interventions in a simulation, clearly distinguishing simulated outcomes from real-world improvements.

15. Roadmap Completion Checklist

- ■ 1. Fundamentals — Build a command-line program using conditions, loops, functions, and collections.
- ■ 2. Intermediate Python — Create a multi-file application that reads and writes persistent data.
- ■ 3. Advanced Python — Write tested, typed, modular code and use Git confidently.
- ■ 4. Backend — Build a FastAPI application with PostgreSQL and authentication.
- ■ 5. APIs — Consume an external API with validation, timeouts, retries, and tests.
- ■ 6. Full-stack — Connect a React frontend to your backend and deploy it.
- ■ 7. Machine learning — Train and evaluate a model using a valid holdout strategy.
- ■ 8. ML serving — Expose predictions through a documented API.
- ■ 9. Production — Deploy with Docker, automated tests, configuration, and monitoring.
- ■ 10. Specialization — Complete a portfolio-quality project and explain its trade-offs.

16. Rules That Will Make You a Better Programmer

1. Code every day you study. Watching tutorials is not the same as learning to program.
2. Build projects at every stage. Do not wait until you finish all the theory.
3. Try before asking AI for code. Attempt the problem, inspect the error, and ask for a hint before asking for a full solution.
4. Understand code you copy. Be able to explain what it does and change it without breaking everything.
5. Learn debugging early. Errors are part of programming, not evidence that you are bad at it.
6. Use Git from the beginning. Commit meaningful changes and keep projects organized.
7. Learn one main technology at a time: Python, backend, frontend, then ML. Overlap them later.
8. Measure project quality. Test correctness, data quality, security, and performance rather than judging only by the interface.

Final Recommendation

Spend your first four weeks on Python fundamentals, then progress into intermediate Python and software engineering. Start backend development before advanced machine learning, and learn enough HTML, CSS, and JavaScript to build a usable interface. Once you have these foundations, use MOBIUS AI as your long-term project to bring everything together.

The goal is more than completing courses: build a portfolio of applications you can explain, test, deploy, and improve independently.

Resource URLs are provided for convenience; check official documentation for current versions and recommended practices.